



دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده برق و کامپیوتر



یادگیری ماشین

تمرین پایانی (فاز دوم)

نام و نام خانوادگی

سید محمد جزایری 810101399

امیرحسین عارفزاده 810101604

محمد سینا پرویزی مطلق 810101394

فہرست شکل‌ها.....	۳
فہرست جدول‌ها.....	۴
چکیده.....	۵
PART 1: EXPLANATION OF PREPROCESSING METHODS	۶
PART 2: FEATURE EXTRACTION.....	۷
PART 3: CLASSIFICATION	۸
PART 4: CLUSTERING	۱۲
PART 5: EVALUATION.....	۱۶
مراجع.....	۲۱

Figure1 . The distribution of the audio files before the split.....	8
Figure2 . The distribution of the audio files after the split.....	8
Figure 3. PCA output.....	9
Figure 4.Shuffle label output	9
Figure5 .KNN stats	10
Figure6 .RBF-SVM output	10
Figure7 . Random forest output	10
Figure8 . Logistic regression output	10
Figure9 . MLP output.....	11
Figure10 . All in one look.....	11
Figure 11. Inertia VS k	12
Figure12 . Silhouette VS k.....	12
Figure13 . Clusters info	13
Figure14 . Visualized clustering	13
Figure15 . Silhouette VS k in agglomerative.....	14
Figure16 . Clusters info	14
Figure 17. Clusters visualized	15
Figure 18. Comparing two algorithms.....	15
Figure19 . Centroid distance (Euclidean)	18
Figure20 . Within-language spread (avg distance to centroid) and Average pairwise distance (sampled Euclidean).....	18

فهرست جدول‌ها

No table of figures entries found.

Part 1: Explanation of preprocessing methods

In the initial phase of the project, raw audio files were subjected to a rigorous preprocessing pipeline to ensure data consistency and quality before feature extraction. The following steps were implemented:

Resampling (22050 Hz): All audio files were resampled to a uniform sample rate of 22,050 Hz. This specific rate was chosen because it captures frequencies up to 11,025 Hz (Nyquist frequency), which covers the essential frequency range of human speech while significantly reducing computational load compared to higher rates like 44.1 kHz.

Silence Removal: We applied a silence trimming technique using a threshold of 25dB (top_db=25). Since the goal is language identification, silence carries no phonetic information and acts as noise. Removing it ensures that the model trains only on active speech segments, preventing the learning of silence patterns that might be artifacts of specific recording environments.

Amplitude Normalization: The audio signals were normalized to fit within the range $[-1,1]$. This step mitigates the issue of varying recording volumes. Without normalization, the model might incorrectly classify languages based on loudness rather than phonetic content (e.g., classifying all quiet recordings as Spanish simply because that specific dataset volume was lower).

Fixed-Length Segmentation (4 Seconds): Instead of using the full 1-minute files, we split the audio into non-overlapping 4-second segments. This was a critical decision for two reasons:

- 1. Data Augmentation:** It drastically increased our dataset size from hundreds of files to over 10,000 samples, preventing overfitting.

- 2. Stationarity:** Speech signals are non-stationary over long periods. By analyzing shorter 4-second windows, we can capture local phonetic properties more effectively than averaging features over a full minute, which would smooth out distinct language characteristics.

Part 2: Feature Extraction

For the feature extraction phase, we moved beyond simple statistical means and extracted a 51-dimensional feature vector for each segment. We calculated both the Mean and Variance for most features to capture not just the average spectral shape, but also the temporal dynamics and variability of the speech.

- **Mel-Frequency Cepstral Coefficients (MFCCs):** We extracted 20 MFCCs (Mean and Variance), resulting in 40 features. MFCCs are the standard for speech processing as they approximate the human auditory system's response. They effectively represent the "timbre" of the voice and the phonetic content, which are the primary discriminators between different languages.
- **Spectral Centroid:** This feature measures the "center of mass" of the spectrum and indicates the brightness of the sound. It helps distinguish languages that may have higher usage of fricatives or brighter vowel sounds.
- **Zero Crossing Rate (ZCR):** ZCR measures the rate at which the signal changes sign. It is particularly useful for distinguishing between voiced (low ZCR) and unvoiced (high ZCR) speech segments, providing clues about the rhythmic structure of the language.
- **Chroma STFT:** Chroma features project the entire spectrum onto 12 bins representing the 12 distinct semitones (musical chroma) of the octave. While often used in music, in speech, this helps capture tonal characteristics and pitch class profiles, which can be relevant for prosody analysis.
- **Spectral Contrast:** This feature computes the difference between peaks and valleys in the spectrum. It ensures the model is robust against noise and provides a high-level description of the spectral texture.

By combining these spectral, temporal, and prosodic features, we created a comprehensive representation of the audio that is robust against speaker gender variations (via Variance features) and distinct enough for multi-class language classification.

Part 3: Classification

In this part we classified the dataset using the 48 features we extracted in the previous part. We divided the data 80/20 as the assignment suggested and ensured that the languages were proportionally distributed as you can see in the image below. The Italians are somewhat more because they were more than the rest, but the other languages are very close in numbers.

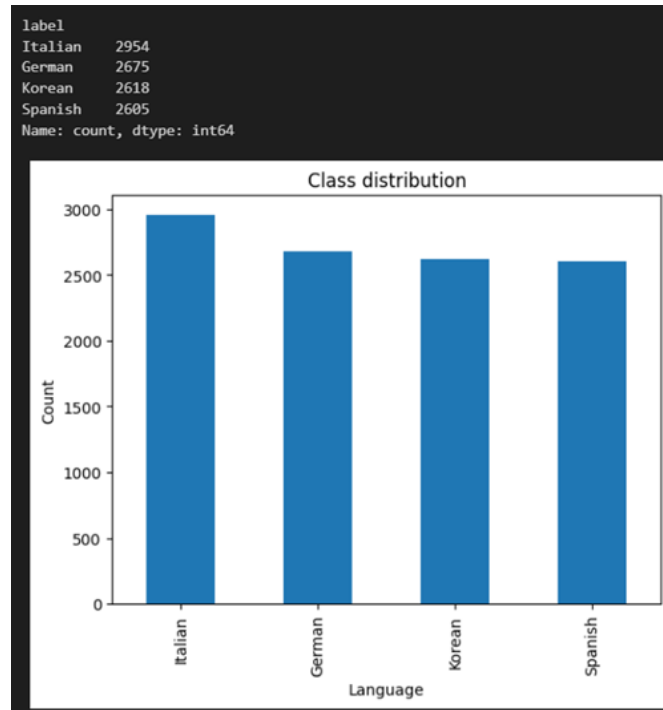


Figure 1. The distribution of the audio files before the split

The image below also ascertains that the proportions of the languages were maintained even after the split so that the data wouldn't be imbalanced, and hence ensure the correctness of the learning process of all the models.

```
Train: (8681, 48) Test: (2171, 48)
Train class distribution:
label
Italian    0.272
German     0.247
Korean     0.241
Spanish    0.240
Name: proportion, dtype: float64
Test class distribution:
label
Italian    0.272
German     0.246
Korean     0.241
Spanish    0.240
Name: proportion, dtype: float64
```

Figure 2. The distribution of the audio files after the split

For classification we used PCA with the condition that 95% of the data distribution would be preserved and as you can see below PCA kept 32 out of the 48 feature which indicates a significant 33% reduction in the feature space, and it is also worth mentioning that for the classification task this increased accuracy unlike with clustering.

```
pca = pipe.named_steps["preprocess"].named_steps.get("pca", None)
if pca is not None:
    print("PCA kept components:", pca.n_components_)
    print("Explained variance kept:", pca.explained_variance_ratio_.sum())
results.append(evaluate_model(name, pipe, X_test, y_test))

results_df = pd.DataFrame(results).sort_values(by="f1_macro", ascending=False)
results_df
```

✓ 16.9s

PCA kept components: 32
Explained variance kept: 0.9522622237963869

Figure 3. PCA output

Next we implemented 5 models, RBF-SVM, MLP, logistic regression, KNN and random forest. We used scikit-learn to implement them and the results were staggering to say the least because we achieved near 100% accuracy with every model and we check for data leakage and learning some feature outside the feature space but nothing turned up and we even shuffled the labels for the training dataset and achieved 25% accuracy which only proves the integrity and correctness of the model.

```
y_shuf = y_train.sample(frac=1.0, random_state=42).reset_index(drop=True)
X_tr = X_train.reset_index(drop=True)

pipe = pipelines["SVM_RBF"]
pipe.fit(X_tr, y_shuf)
pred = pipe.predict(X_test)

from sklearn.metrics import accuracy_score
print("Shuffled-label accuracy:", accuracy_score(y_test, pred))
```

✓ 5.1s

Shuffled-label accuracy: 0.24044219253800092

Figure 4. Shuffle label output

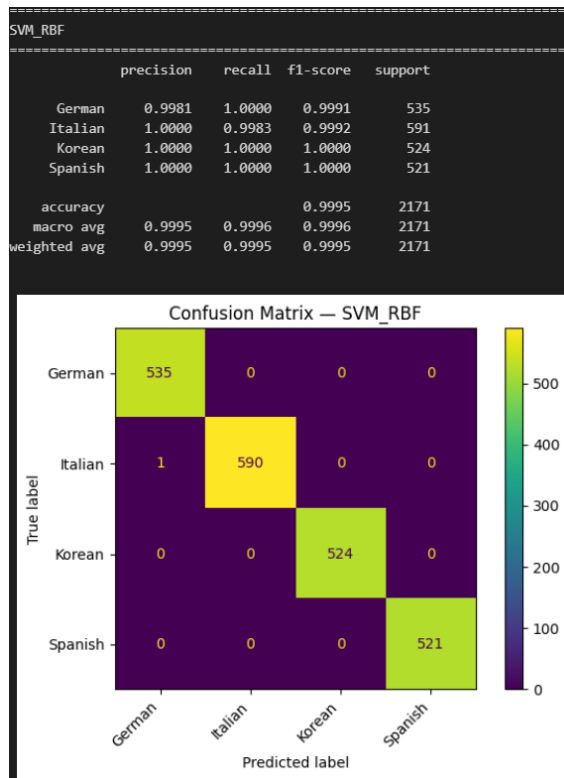


Figure 6. RBF-SVM output

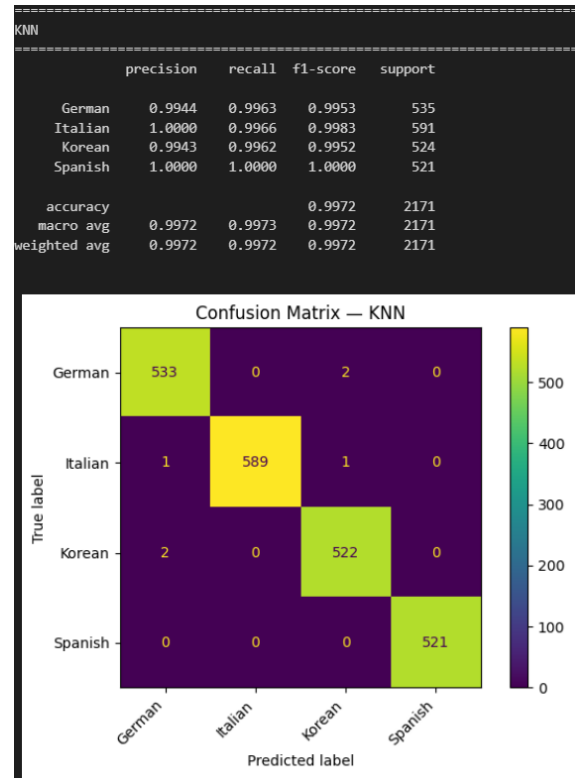


Figure 5. KNN stats

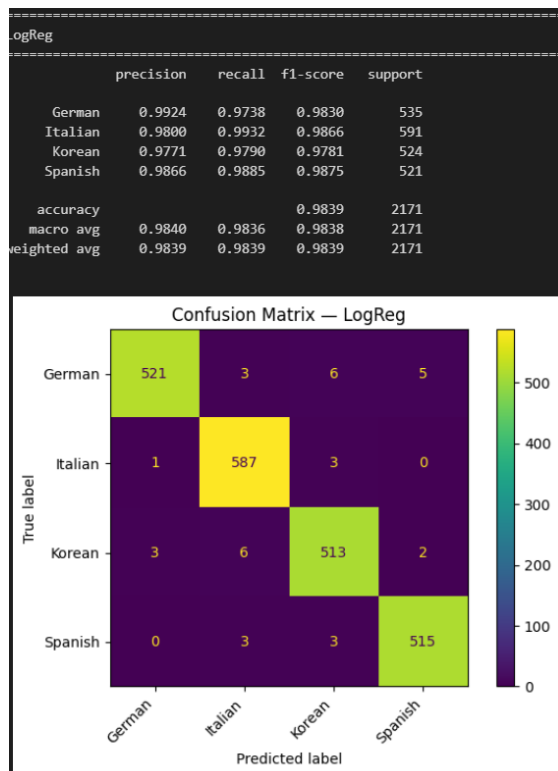


Figure 8. Logistic regression output

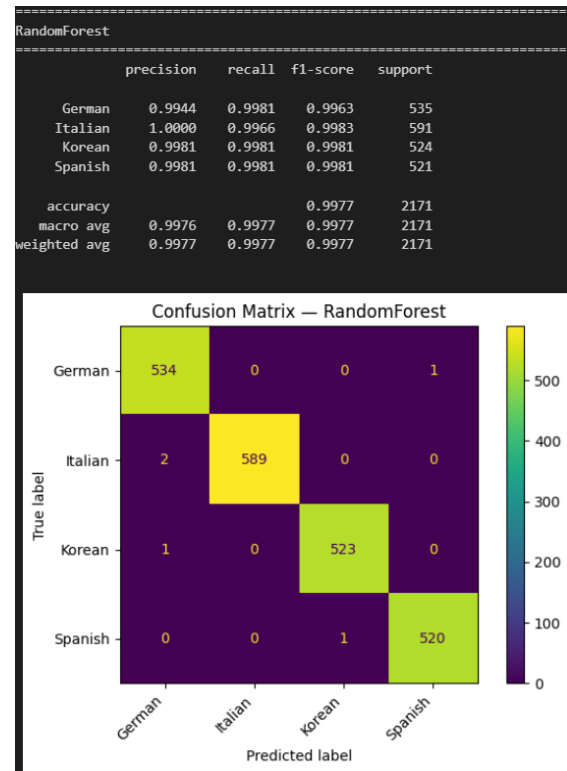


Figure 7. Random forest output

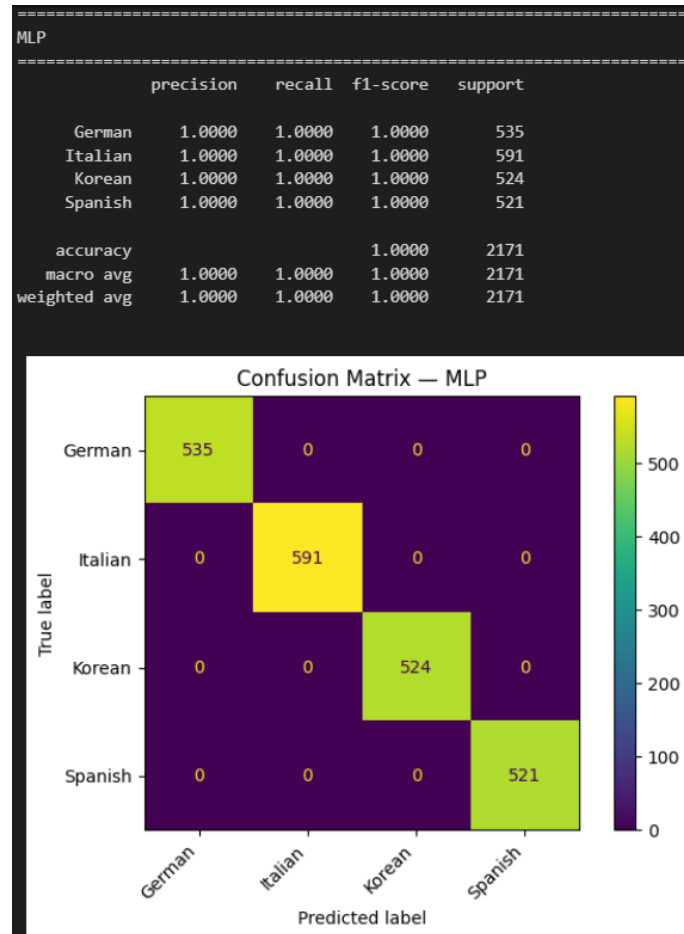


Figure9. MLP output

model	accuracy	f1_macro	precision_macro	recall_macro	f1_weighted
MLP	1.000000	1.000000	1.000000	1.000000	1.000000
SVM_RBF	0.999539	0.999555	0.999534	0.999577	0.999539
RandomForest	0.997697	0.997686	0.997646	0.997730	0.997698
KNN	0.997236	0.997218	0.997172	0.997265	0.997238
LogReg	0.983878	0.983796	0.984020	0.983639	0.983872

Figure10. All in one look

It is worth mentioning that without PCA SVM was the best but after PCA MLP reached 100% accuracy and logistic regression was the worst in both cases.

Part 4: Clustering

In this part we clustered the dataset with K-means and agglomerative algorithms. For the variable k We chose the range from 2 to 11. But we tried more than 11 even up to 20 and the scores kept rising so we decided to stop at 11. In the images below we will visualize the aforementioned argument. Furthermore the scores that we will discuss in the evaluation part also increased with the increase of .

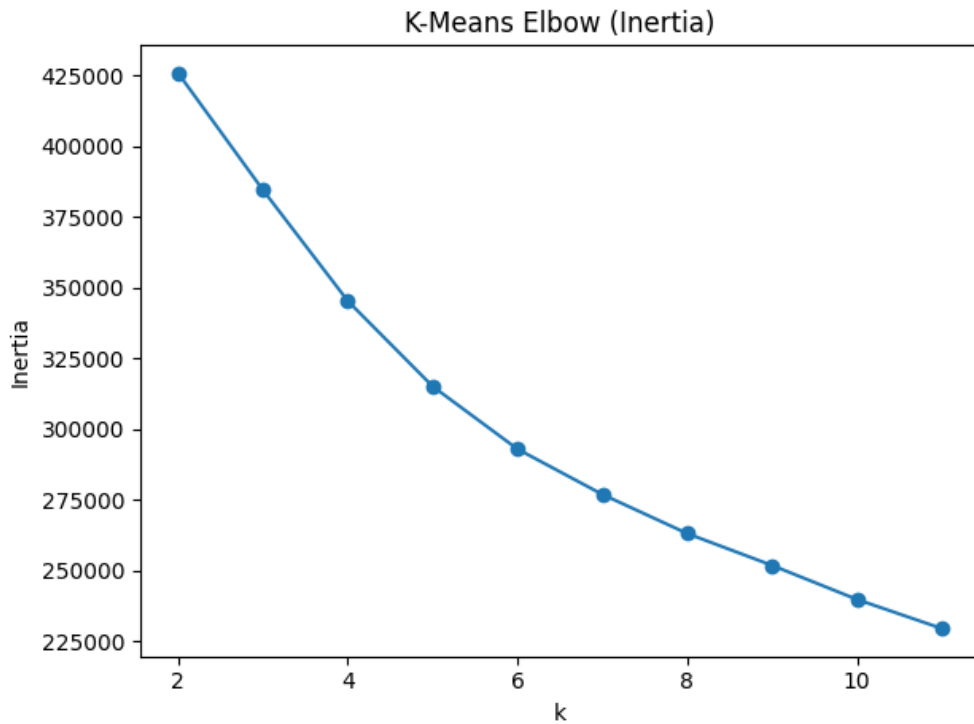


Figure 11. Inertia VS k

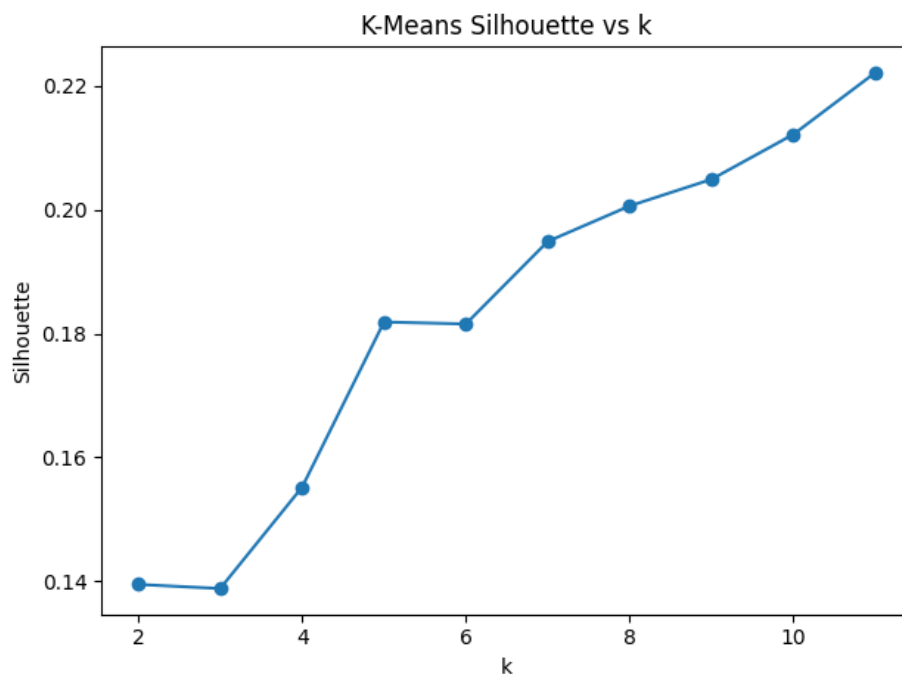


Figure 12. Silhouette VS k

label	German	Italian	Korean	Spanish
cluster				
0	688	4	1323	6
1	677	9	9	436
2	44	1284	420	1
3	0	0	0	1290
4	0	965	0	0
5	687	0	0	0
6	0	584	0	0
7	0	0	865	0
8	579	1	0	0
9	0	3	1	872
10	0	104	0	0

Figure13. Clusters info

As we can see in the image above, except for the 3 first clusters the rest seem highly homogeneous and that is a good sign.

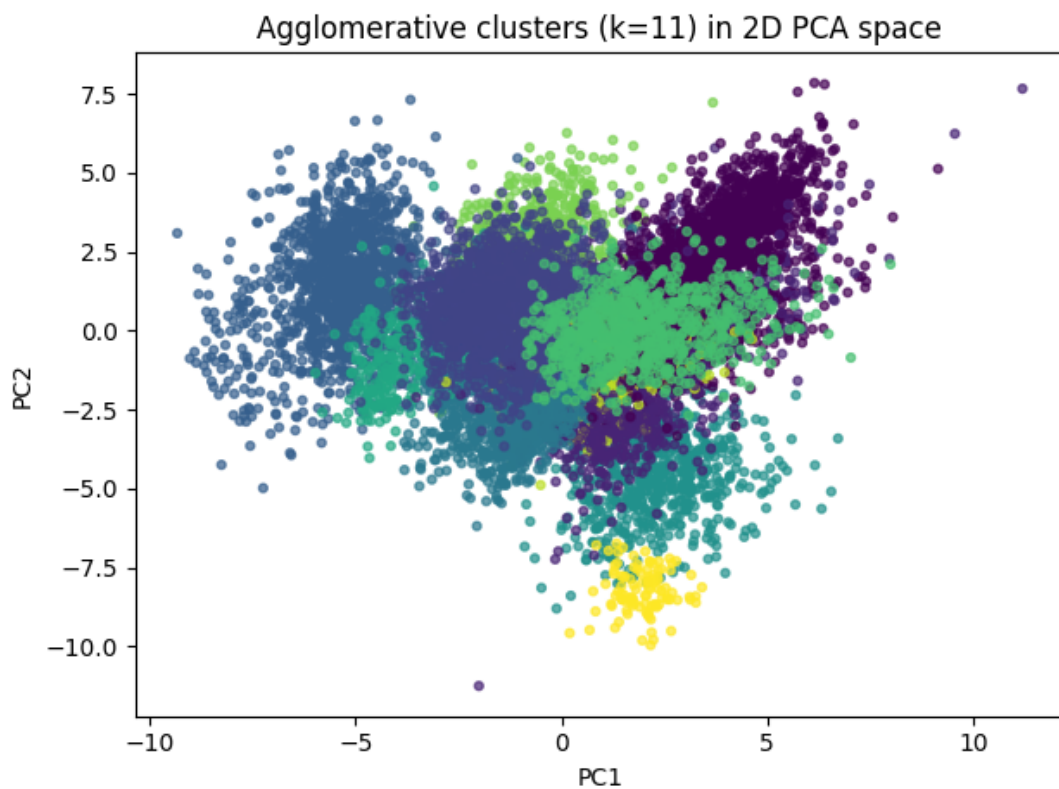


Figure14. Visualized clustering

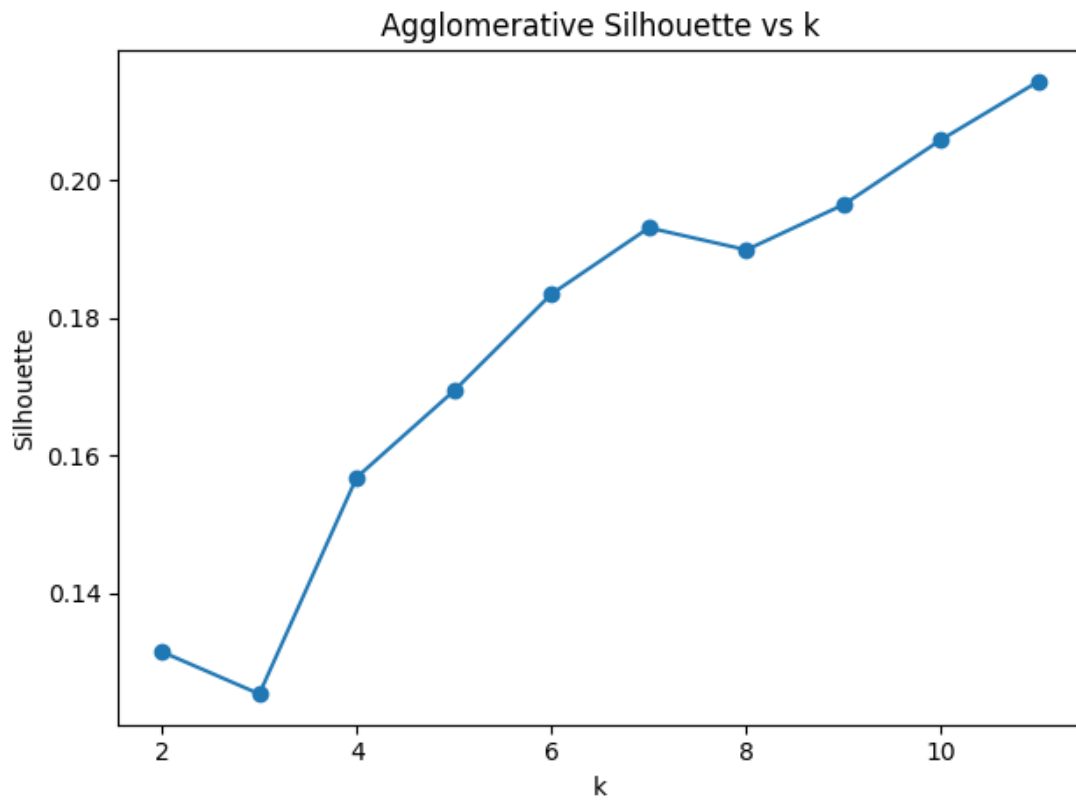


Figure15. Silhouette VS k in agglomerative

label	German	Italian	Korean	Spanish
cluster				
0	688	4	1323	6
1	677	9	9	436
2	44	1284	420	1
3	0	0	0	1290
4	0	965	0	0
5	687	0	0	0
6	0	584	0	0
7	0	0	865	0
8	579	1	0	0
9	0	3	1	872
10	0	104	0	0

Figure16. Clusters info

We can see that the two algorithms behaved fairly similarly, with the same chosen k.

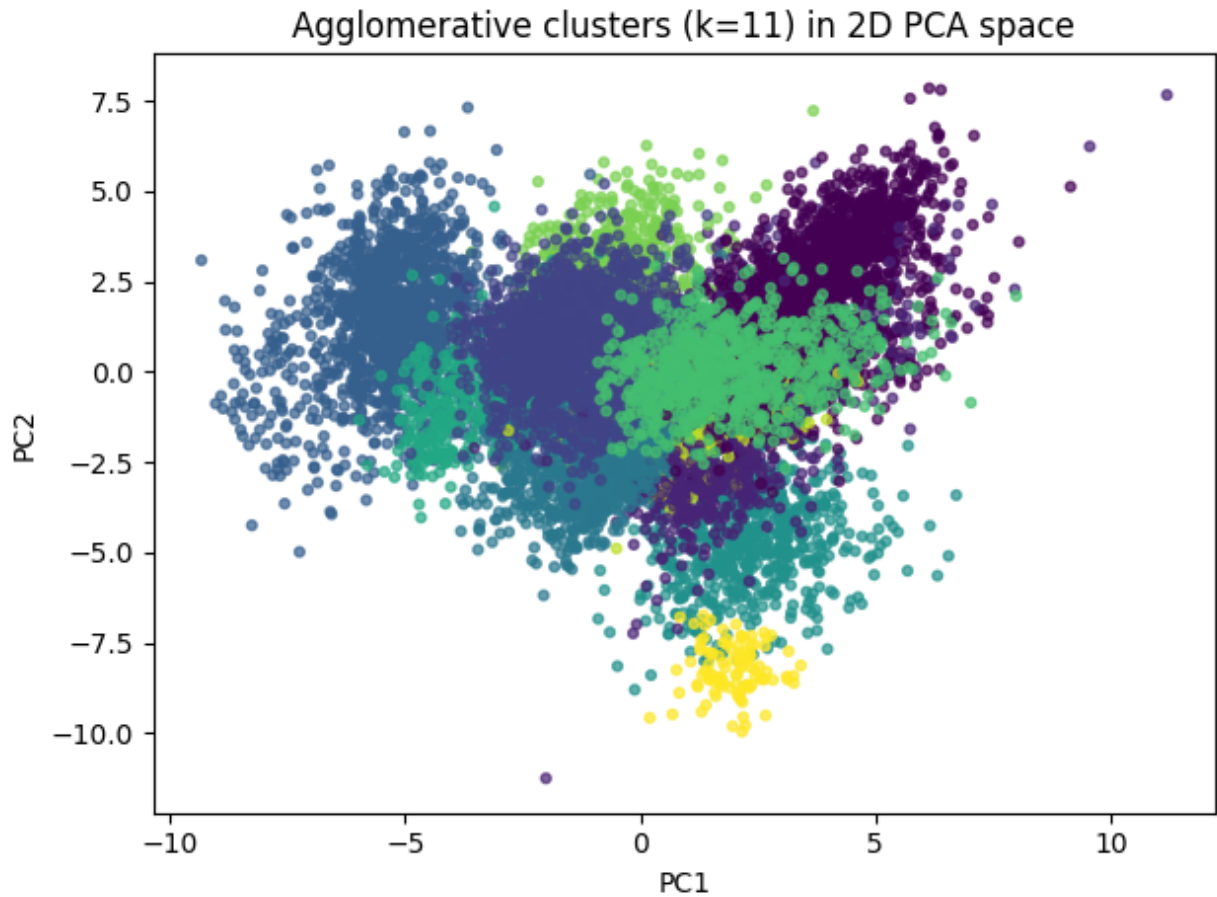


Figure 17. Clusters visualized

	algorithm	silhouette	calinski_harabasz	davies_bouldin	purity
0	KMeans (k=11)	0.222257	1259.777074	1.59066	0.851548
1	Agglomerative (k=11)	0.214448	1191.457036	1.55706	0.850534

Figure 18. Comparing two algorithms

It is clear from the 2D visualization and the clusters info that each little cluster includes a number of the audio files of a specific language; this could be because partitioned each audio file into smaller 3-4 seconds sub-sections and some of these might, due to their small length, consider very similar vocal patterns that belong to that very specific language and these clusters represent those patterns.

Part 5: Evaluation

Clustering:

What each score represents let's walk through them one by one.

Silhouette:

For a point x_i :

- **Intra-cluster distance**

$$a(i) = \frac{1}{(|C_{c(i)}| - 1)} \times \sum_{x_j \in C_{c(i)}, j \neq i} d(x_i, x_j)$$

- **Nearest-cluster distance**

$$b(i) = \min_{k \neq c(i)} \left(\frac{1}{|C_k|} \times \sum_{x_j \in C_k} d(x_i, x_j) \right)$$

- **Silhouette of point i**

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- **Overall silhouette**

$$S = \frac{1}{n} \sum_{i=0}^n s(i)$$

Measures how well-separated clusters are, using only distances:

- close to **1**: points are much closer to their own cluster than others (strong separation)
- around **0**: clusters overlap / boundaries are fuzzy
- **< 0**: many points are closer to another cluster than their assigned one (bad clustering)

Calinski–Harabasz:

In the course we had within scatter(SW) and between scatter(SB). This metric is defined as:

$$CH = \frac{S_B / (K - 1)}{S_W / (n - K)}$$

This metric is unbounded. Ratio of between-cluster dispersion to within-cluster dispersion.

Higher means clusters are more separated and compact *in aggregate*. It tends to increase with more clusters, so it's best used for comparing models/k on the same dataset (not as an absolute good/bad number).

Davies–Bouldin:

Average similarity of each cluster with its most similar other cluster (cluster scatter / cluster separation).

Lower means clusters are tighter and farther apart.

We define the separation between two clusters as the absolute value of the difference of their means.

$$M_{ij} = ||\mu_i - \mu_j||$$

Cluster-Pair Similarity:

$$R_{ij} = \frac{S_W(i) + S_W(j)}{M_{ij}}$$

For each cluster i , find its worst-case (most similar) neighbor:

$$R_i = \max_{j \neq i} R_{ij}$$

Then the **Davies–Bouldin index** is:

$$DB = \frac{1}{K} \sum_{i=1}^K R_i$$

Lower is better.

Purity:

Let ground-truth labels be $(y_i \in \{1, \dots, J\})$.

Define $(L_j = \{x_i : y_i = j\})$.

For each cluster (C_k) , count how many points belong to each true class and take the maximum:

$$\text{Purity} = \frac{1}{n} \sum_{k=1}^K \max_j |C_k \cap L_j|$$

Closer to 1 means clusters are more “single-label,” but purity increases as K increases, so it should be interpreted alongside K .

The silhouette scores (~ 0.21 – 0.22) are relatively low, indicating that the data does not form very clean, well-separated “blobs” in feature space. However, purity is high (~ 0.85), meaning that many clusters are dominated by a single language.

The fact that the best internal clustering configuration uses $k = 11$ (not $k = 4$) suggests that the dataset contains meaningful substructure beyond language identity, such as:

- speaker differences,
- recording/channel conditions,
- background noise or speaking style.

This explains why supervised classifiers can still achieve near-perfect language classification: classification can learn decision boundaries even when clusters overlap, while clustering methods require compact, separable groups to achieve high silhouette.

Between the two methods, K-Means slightly outperforms Agglomerative on silhouette and Calinski–Harabasz, while Agglomerative has a slightly better (lower) Davies–Bouldin score; overall their clustering quality is very similar.

To help us understand better we calculate the distance between languages and we'll paste them below and analyze them.

Centroid distance matrix (Euclidean):

	German	Italian	Korean	Spanish
German	0.000000	4.840625	3.582130	4.964155
Italian	4.840625	0.000000	5.221604	3.928424
Korean	3.582130	5.221604	0.000000	5.594354
Spanish	4.964155	3.928424	5.594354	0.000000

Figure19. Centroid distance (Euclidean)

Within-language spread (avg distance to centroid):

	language	avg_dist_to_centroid	n
3	Spanish	6.398715	2605
0	German	6.088090	2675
1	Italian	6.079690	2954
2	Korean	5.096088	2618

Average pairwise distance matrix (sampled Euclidean):

	German	Italian	Korean	Spanish
German	0.000000	9.939273	8.606397	10.136695
Italian	9.939273	0.000000	9.544818	9.773103
Korean	8.606397	9.544818	0.000000	9.935589
Spanish	10.136695	9.773103	9.935589	0.000000

Figure20. Within-language spread (avg distance to centroid) and Average pairwise distance (sampled Euclidean)

Smaller values indicate two languages have more similar average acoustic feature patterns.

Centroid distance matrix (Euclidean):

- Closest pair: German–Korean = 3.58

- Second closest: Italian–Spanish = 3.93
- Larger distances (more separated):
 - German–Spanish = 4.96
 - Italian–Korean = 5.22
 - Korean–Spanish = 5.59

Interpretation:

The feature space shows two “near pairs”: German is closest to Korean, and Italian is closest to Spanish. This suggests that if clustering merges languages, it would most likely mix those pairs first.

We computed the average distance of samples to their own language centroid (a measure of how internally spread out each language is in feature space).

Average distance to centroid (higher = more variable):

- Spanish: 6.40 (n=2605)
- German: 6.09 (n=2675)
- Italian: 6.08 (n=2954)
- Korean: 5.10 (n=2618)

Interpretation:

Spanish, German, and Italian are relatively more dispersed, while Korean is more compact. Higher spread can be caused by speaker variability, recording differences, or broader acoustic diversity within that language subset.

Centroid distances only compare means. To compare the overall distributions, we computed the mean distance between randomly sampled points from two languages (average pairwise Euclidean distance). These values are larger than centroid distances because they include within-language variation.

Average pairwise distance matrix:

- Closest pair: German–Korean = 8.61
- Next: Italian–Korean = 9.54
- Italian–Spanish = 9.77
- German–Italian = 9.94
- Korean–Spanish = 9.94
- Farthest pair: German–Spanish = 10.14

Interpretation:

The pairwise results confirm the centroid result: German–Korean are closest overall, and German–Spanish are the most separated. However, all between-language pairwise distances are in a relatively narrow range (~ 8.6 – 10.1), indicating that between-language separation is not dramatically larger than within-language variation.

Our clustering evaluation showed a low silhouette (~ 0.22) but relatively high purity (~ 0.85). The distance analysis explains this:

- **Silhouette is low** because clusters are not cleanly separated by distances: within-language spreads (~ 5 – 6.4) are not tiny compared to between-language distances (centroid ~ 3.6 – 5.6 ; pairwise ~ 8.6 – 10.1). This creates overlap and fuzzy boundaries, which silhouette penalizes.
- **Purity is high** because languages still have structure: even with overlap, clusters often capture language-dominant regions, especially when using more clusters ($k=11$) to model subgroups (e.g., speaker/channel effects). That produces clusters that are label-dominated without requiring perfectly separated 4 blobs.

Classification: Arguments are made with reference to figure 10.

Spanish is consistently classified extremely well (often perfect precision/recall). The small number of remaining errors for the strongest models (SVM/MLP) occur mainly among German / Italian / Korean, but rates are still extremely low.

Logistic Regression performs the worst, which is expected since it is linear and less expressive than nonlinear models (SVM/MLP).

Overall performance is very high for all models except logistic regression, indicating that the extracted MFCC + spectral features contain strong language-discriminative information. The best-performing model under PCA was the MLP (100% on the test set), closely followed by SVM (RBF) (99.95%).

